

上机实验4：Cache性能分析

一、实验目的

- (1) 加深对cache大小、相联度等对cache性能的影响的理解；
- (2) 掌握Cache模拟器的设计与实现方法；
- (3) 学习使用不同参数组合测试Cache性能；
- (4) 掌握Cache性能分析方法。

二、实验平台及内容

在本次实验中，选择自己设计一个Cache模拟器，并实现了以下功能：

- A. 能够模拟Cache的执行过程；
- B. 支持图形交互；
- C. 支持以下参数配置：
 - trace_input_file：输入文件
 - cache_size：Cache大小（8KB-64KB）
 - cache_associativity：相联度（1-8路）
 - cache_block_size：块大小（16B-128B）
- D. 使用LRU替换算法和写分配策略；
- E. 支持4个trace文件的测试；
- F. 提供详细的Cache性能统计。

三、模拟器设计思想

1. 总体分析与设计

任务分析

目标是实现一个Cache模拟器，能够模拟不同参数配置下Cache的访问过程，并统计性能指标。模拟器需要支持多种Cache参数组合，能够处理不同类型的访问（读数据、写数据、取指令），并实现LRU替换算法。

设计思路

- **CacheSimulatorGUI类**：图形界面类，负责用户交互和结果显示
- **Visualization类**：数据可视化类，负责生成性能分析图表
- **Cache类**：核心类，实现Cache的基本功能

2. 主要类与功能说明

CacheSimulatorGUI类

```
class CacheSimulatorGUI:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("Cache模拟器")
        # 初始化GUI组件
        self.setup_gui()
```

功能：

- 提供图形界面进行参数配置
- 支持文件选择
- 显示模拟结果
- 生成性能分析图表

Visualization类

```
class CacheVisualizer:
    def __init__(self):
        self.font = FontProperties(family='Microsoft YaHei')
```

功能：

- 生成参数组合表格
- 绘制读写块数量统计图
- 绘制失效率统计图
- 支持多种图表样式

Cache类

```
class Cache:
    def __init__(self, cache_size, block_size, associativity):
        self.cache_size = cache_size # 字节
        self.block_size = block_size # 字节
        self.associativity = associativity # 路数

        # 计算Cache参数
        self.num_blocks = cache_size // block_size
        self.num_sets = self.num_blocks // associativity
        self.offset_bits = int(math.log2(block_size))
        self.index_bits = int(math.log2(self.num_sets))
        self.tag_bits = 32 - self.offset_bits - self.index_bits

        # 初始化Cache
        self.cache = [OrderedDict() for _ in range(self.num_sets)]

        # 统计信息
        self.stats = {
            'total_accesses': 0,
            'total_misses': 0,
            'instruction_accesses': 0,
            'instruction_misses': 0,
            'data_reads': 0,
            'data_read_misses': 0,
            'data_writes': 0,
            'data_write_misses': 0,
            'replacements': 0
        }
```

功能：

- 初始化Cache参数（大小、块大小、相联度）
- 计算Cache结构参数（组数、偏移位数、索引位数、标签位数）
- 使用OrderedDict实现LRU替换算法
- 维护访问统计信息

关键方法：

1. access：处理Cache访问
2. get_set_index：计算组索引
3. get_tag：计算标签
4. get_statistics：获取统计信息

3. 关键实现思路

地址解析与Cache结构

首先，Cache模拟器需要将32位地址分解为三个部分：标签（tag）、组索引（index）和块偏移（offset）。这是通过位运算实现的：

```
def get_set_index(self, address):
    return (address >> self.offset_bits) & ((1 << self.index_bits) - 1)

def get_tag(self, address):
    return address >> (self.offset_bits + self.index_bits)
```

例如，对于一个8KB的Cache，64B的块大小，4路组相联：

- 块偏移位： $\log_2(64) = 6$ 位
- 组索引位： $\log_2(8KB/64B/4) = 5$ 位
- 标签位： $32 - 6 - 5 = 21$ 位

组相联Cache的组织结构

Cache被组织为一个数组，每个元素是一个组（set），每个组使用OrderedDict实现：

```
self.cache = [OrderedDict() for _ in range(self.num_sets)]
```

这种结构使得每个组可以独立管理其缓存行，同时保持访问顺序。OrderedDict是Python标准库collections模块中的一个特殊字典类，它继承自普通的dict类，但有一个重要的特性：保持键值对的插入顺序。OrderedDict的键是标签（tag），值是一个布尔值（表示该行是否有效）。

LRU替换算法的实现

LRU（最近最少使用）算法的核心思想是：当需要替换时，选择最久未使用的缓存行进行替换。使用OrderedDict可以实现这一点：

```
def access(self, access_type, address):
    set_index = self.get_set_index(address)
    tag = self.get_tag(address)
    cache_set = self.cache[set_index]

    # 检查是否命中
    if tag in cache_set:
        # 命中时，将该项移到末尾，表示最近使用
        cache_set.move_to_end(tag)
        return True

    # 未命中处理
    if len(cache_set) >= self.associativity:
        # 组已满，需要替换
        # 删除最前面的项（最久未使用）
        cache_set.popitem(last=False)

    # 添加新项到末尾
    cache_set[tag] = True
    return False
```

具体工作流程示例

假设我们有一个4路组相联的Cache，某个组当前状态为：

```
OrderedDict([('tag1', True), ('tag2', True), ('tag3', True), ('tag4', True)])
```

当访问不同标签时：

1. 访问'tag2'（命中）：

```
cache_set.move_to_end('tag2')  
# 结果: OrderedDict([('tag1', True), ('tag3', True), ('tag4', True), ('tag2', True)])
```

2. 访问'tag5'（未命中，组已满）：

```
cache_set.popitem(last=False) # 删除'tag1'  
cache_set['tag5'] = True  
# 结果: OrderedDict([('tag3', True), ('tag4', True), ('tag2', True), ('tag5', True)])
```

性能统计的实现

每次访问都会更新相应的统计信息：

```
def access(self, access_type, address):  
    self.stats['total_accesses'] += 1  
  
    # 更新访问类型统计  
    if access_type == 2: # 指令访问  
        self.stats['instruction_accesses'] += 1  
    elif access_type == 0: # 数据读  
        self.stats['data_reads'] += 1  
    elif access_type == 1: # 数据写  
        self.stats['data_writes'] += 1  
  
    # ... 访问处理 ...  
  
    # 未命中统计  
    if tag not in cache_set:  
        self.stats['total_misses'] += 1  
        if access_type == 2:  
            self.stats['instruction_misses'] += 1  
        elif access_type == 0:  
            self.stats['data_read_misses'] += 1  
        elif access_type == 1:  
            self.stats['data_write_misses'] += 1
```

命中率计算

最后，通过统计信息计算各种访问类型的命中率：

```
def get_statistics(self):
    stats = self.stats.copy()

    # 计算各种命中率
    stats['total_hit_rate'] = 1 - (stats['total_misses'] / stats['total_accesses']) if stats['total_accesses'] > 0 else 0

    stats['instruction_hit_rate'] = 1 - (stats['instruction_misses'] / stats['instruction_accesses']) if stats['instruction_accesses'] > 0 else 0

    stats['data_read_hit_rate'] = 1 - (stats['data_read_misses'] / stats['data_reads']) if stats['data_reads'] > 0 else 0

    stats['data_write_hit_rate'] = 1 - (stats['data_write_misses'] / stats['data_writes']) if stats['data_writes'] > 0 else 0

    return stats
```

四、实验结果分析

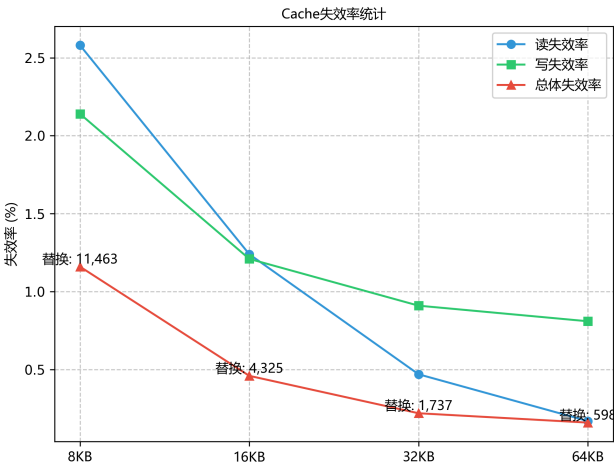
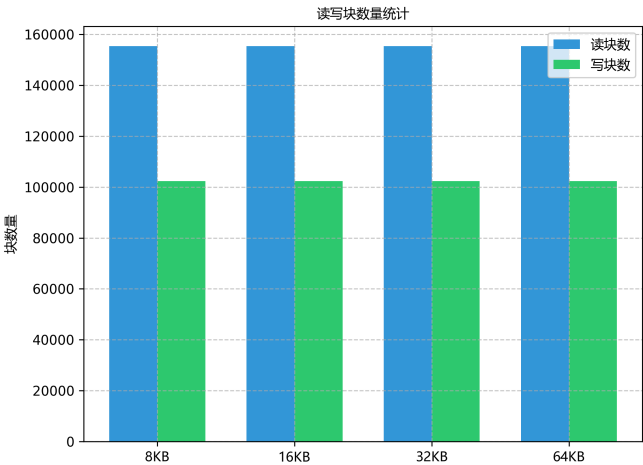
对照教材给出的cache模拟器，发现模拟成功，实验成功。
完整的测试参数与结果见附录。

1. Cache大小对性能的影响

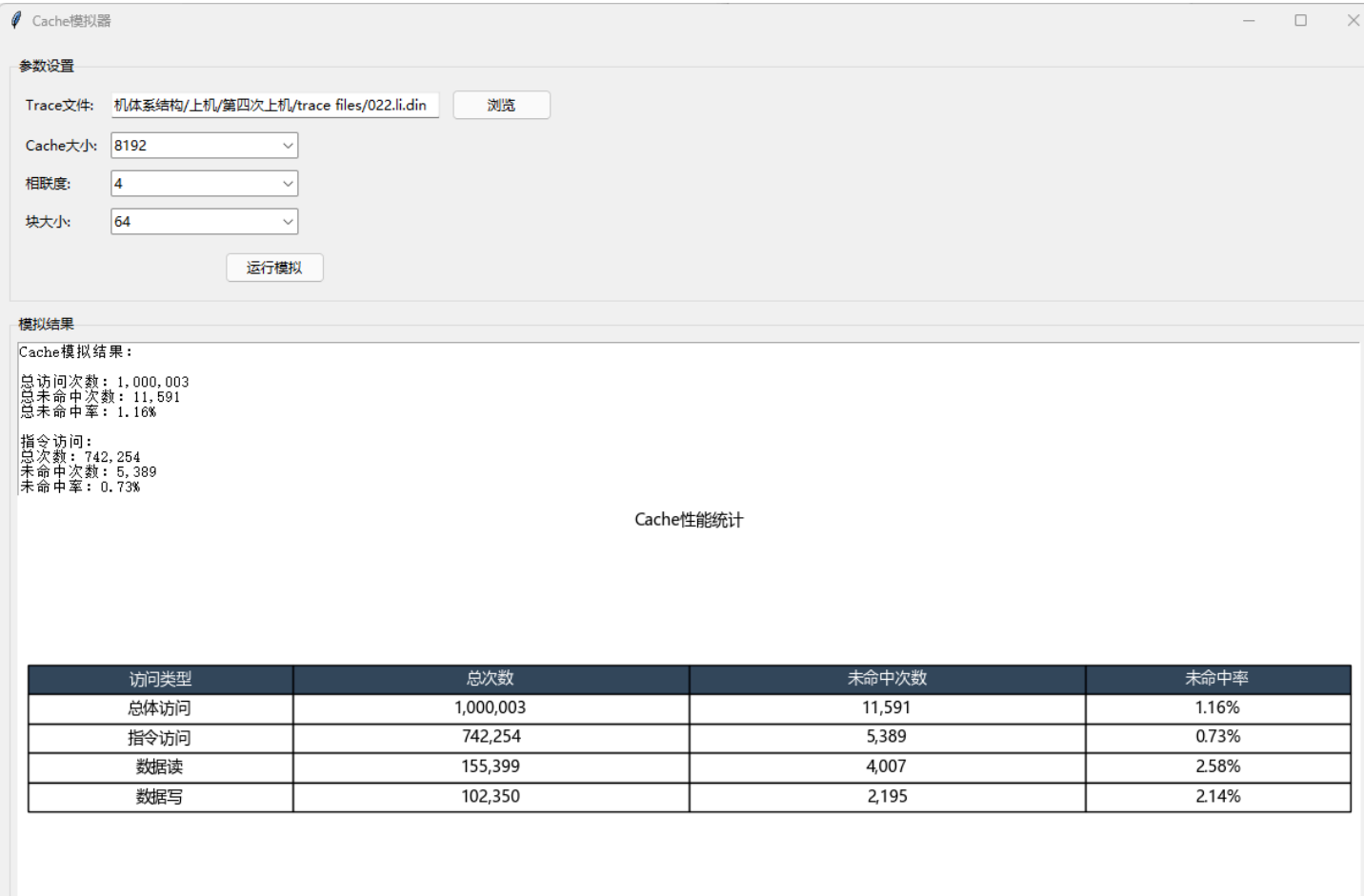
参数组合与效果见图表：

测试参数组合

Cache大小	相联度	块大小	Trace文件
8KB	4路	64B	022.li.din
16KB	4路	64B	022.li.din
32KB	4路	64B	022.li.din
64KB	4路	64B	022.li.din



给出其中一个参数组合结果示例：



- 1. 总体失效率从1.16%降低到0.16%
- 2. 读失效率从2.58%降低到0.17%
- 3. 写失效率从2.14%降低到0.81%
- 4. 替换次数从11,463减少到598

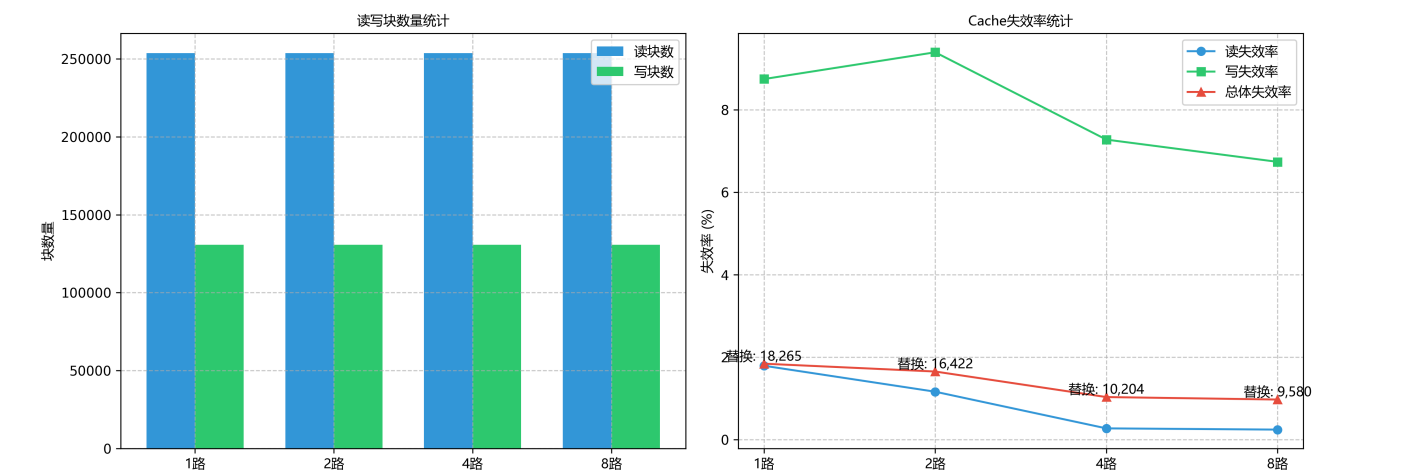
结论：

Cache大小增加能显著降低失效率。这是很符合直觉的，如果cache足够大，失效率将趋近于零。然而，虽然失效率都在降低，但可以发现降低的幅度随着Cache大小的增加而减小。这是由于当cache足够大时，已经能够捕获到大部分局部性访问了，此时cache大小对局部性利用的提升就会越来越小。

2. 相联度对性能的影响

参数组合与效果见图表：

Cache大小	相联度	块大小	Trace文件
8KB	1路	64B	047.tomcatv.din
8KB	2路	64B	047.tomcatv.din
8KB	4路	64B	047.tomcatv.din
8KB	8路	64B	047.tomcatv.din



给出其中一个参数组合结果示例：

Cache模拟器

参数设置

Trace文件: 体系结构/上机/第四次上机/trace files/047.tomcatv. 浏览

Cache大小: 8192

相联度: 1

块大小: 64

运行模拟

模拟结果

Cache模拟结果：

总访问次数：1,000,000
总未命中次数：18,393
总未命中率：1.84%

指令访问：
总次数：615,544
未命中次数：2,411
未命中率：0.39%

Cache性能统计

访问类型	总次数	未命中次数	未命中率
总体访问	1,000,000	18,393	1.84%
指令访问	615,544	2,411	0.39%
数据读	253,723	4,543	1.79%
数据写	130,733	11,439	8.75%

- 1. 总体失效率从1.84%降低到0.97%
- 2. 读失效率从1.79%降低到0.24%

- 3. 写失效率从8.75%降低到6.74%
- 4. 替换次数从18,265减少到9,580

结论：

相联度增加能降低失效率，但效果不如增加Cache大小明显。这是因为相联度主要解决的是冲突失效，对具有良好局部性的程序效果才更明显，只是改变了映射方式，自然没有增加cache大小效果好。

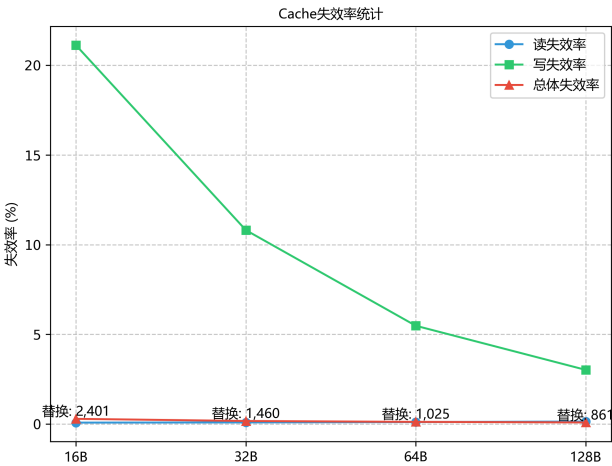
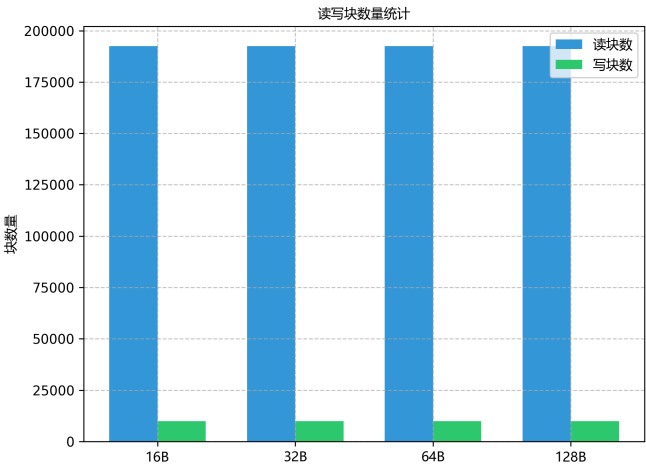
此外，读操作改善效果显著优于写效果，这可能是因为读操作通常比写操作的局部性更好导致的。

3. 块大小对性能的影响

参数组合与效果见图表：

测试参数组合

Cache大小	相联度	块大小	Trace文件
8KB	4路	16B	078.swm256.din
8KB	4路	32B	078.swm256.din
8KB	4路	64B	078.swm256.din
8KB	4路	128B	078.swm256.din



给出其中一个参数组合结果示例：

参数设置

Trace文件: 体系结构/上机/第四次上机/trace files/078.swm256

浏览

Cache大小: 8192

相联度: 4

块大小: 16

运行模拟

模拟结果

Cache模拟结果:
总访问次数: 1,000,000
总未命中次数: 2,913
总未命中率: 0.29%

指令访问:
总次数: 797,647
未命中次数: 674
未命中率: 0.08%

Cache性能统计

访问类型	总次数	未命中次数	未命中率
总体访问	1,000,000	2,913	0.29%
指令访问	797,647	674	0.08%
数据读	192,459	149	0.08%
数据写	9,894	2,090	21.12%

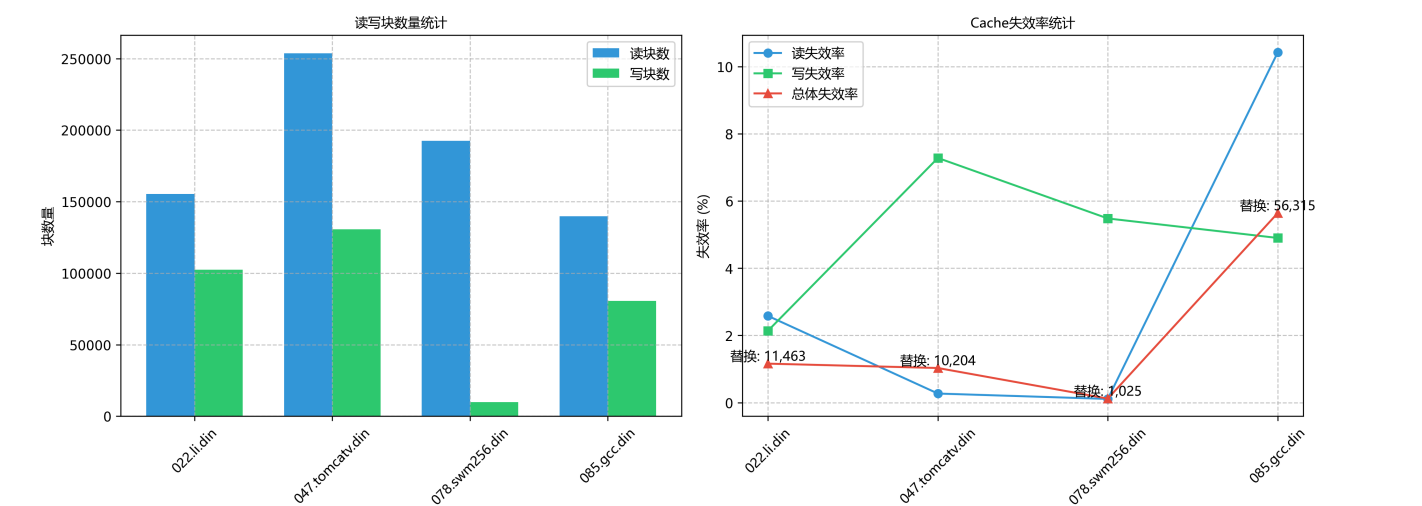
- 1. 总体失效率从0.29%降低到0.09%
- 2. 读失效率从0.08%增加到0.14%
- 3. 写失效率从21.12%降低到3.02%
- 4. 替换次数从2,401减少到861

结论：
块大小增加显著降低了写失效率。这是因为，大块可以一次性写入更多数据，减少写操作的次数。同时，替换次数也显著降低。

4. 不同Trace文件的结果

测试参数组合

Cache大小	相联度	块大小	Trace文件
8KB	4路	64B	022.li.din
8KB	4路	64B	047.tomcatv.din
8KB	4路	64B	078.swm256.din
8KB	4路	64B	085.gcc.din



给出其中一个程序结果示例：

Cache模拟器

参数设置

Trace文件: 体系结构/上机/第四次上机/trace files/085.gcc.din 浏览

Cache大小: 8192

相联度: 4

块大小: 64

运行模拟

模拟结果

Cache性能统计

访问类型	总次数	未命中次数	未命中率
总体访问	1,000,000	56,443	5.64%
指令访问	779,515	37,918	4.86%
数据读	139,816	14,573	10.42%
数据写	80,669	3,952	4.90%

1. 085.gcc.din 的总体失效率最高 (5.64%)
2. 078.swm256.din 的总体失效率最低 (0.12%)

结论：不同Trace文件的读写比例差异很大，同时访问的局部性也有差异。根据四个结果的差异程度，我们可以发现不同程序的访问模式对Cache性能有显著影响。

五、实验感悟

通过本次Cache模拟器的设计与实现，我对Cache的工作原理和性能影响因素有了更深入的理解。实验过程中，我不仅实现了基本的Cache功能，还通过可视化工具分析了各种参数对性能的影响。通过本次实验，我深入理解了Cache的基本概念和实现方法，掌握了LRU替换算法的实现。学会了分析Cache性能的方法，理解了不同参数对Cache性能的影响。

六、附录

参数组合与完整结果：

相联度均为4，cache大小均为8B，块大小均为64

1.022.li.din

Cache模拟结果：

总访问次数：1,000,003总未命中次数：11,591总未命中率：1.16%
指令访问：总次数：742,254未命中次数：5,389未命中率：0.73%
数据读：总次数：155,399未命中次数：4,007未命中率：2.58%
数据写：总次数：102,350未命中次数：2,195未命中率：2.14%
替换次数：11,463

2.047.tomcatv.din

Cache模拟结果：

总访问次数：1,000,000总未命中次数：10,332总未命中率：1.03%
指令访问：总次数：615,544未命中次数：130未命中率：0.02%
数据读：总次数：253,723未命中次数：687未命中率：0.27%
数据写：总次数：130,733未命中次数：9,515未命中率：7.28%
替换次数：10,204

3.078.swm256.din

Cache模拟结果：

总访问次数：1,000,000总未命中次数：1,153总未命中率：0.12%
指令访问：总次数：797,647未命中次数：398未命中率：0.05%
数据读：总次数：192,459未命中次数：213未命中率：0.11%
数据写：总次数：9,894未命中次数：542未命中率：5.48%
替换次数：1,025

4.085.gcc.din

Cache模拟结果：

总访问次数：1,000,000总未命中次数：56,443总未命中率：5.64%
指令访问：总次数：779,515未命中次数：37,918未命中率：4.86%
数据读：总次数：139,816未命中次数：14,573未命中率：10.42%
数据写：总次数：80,669未命中次数：3,952未命中率：4.90%
替换次数：56,315

相联度均为4，Cache大小均为8B，文件选取078.swm256.din

1.块大小16

Cache模拟结果：

总访问次数：1,000,000总未命中次数：2,913总未命中率：0.29%
指令访问：总次数：797,647未命中次数：674未命中率：0.08%
数据读：总次数：192,459未命中次数：149未命中率：0.08%
数据写：总次数：9,894未命中次数：2,090未命中率：21.12%
替换次数：2,401

2.块大小32

Cache模拟结果：

总访问次数：1,000,000总未命中次数：1,716总未命中率：0.17%
指令访问：总次数：797,647未命中次数：474未命中率：0.06%
数据读：总次数：192,459未命中次数：172未命中率：0.09%
数据写：总次数：9,894未命中次数：1,070未命中率：10.81%
替换次数：1,460

3.块大小64

Cache模拟结果：

总访问次数：1,000,000总未命中次数：1,153总未命中率：0.12%
指令访问：总次数：797,647未命中次数：398未命中率：0.05%
数据读：总次数：192,459未命中次数：213未命中率：0.11%
数据写：总次数：9,894未命中次数：542未命中率：5.48%
替换次数：1,025

4.块大小128

Cache模拟结果：

总访问次数：1,000,000总未命中次数：925总未命中率：0.09%
指令访问：总次数：797,647未命中次数：360未命中率：0.05%
数据读：总次数：192,459未命中次数：266未命中率：0.14%
数据写：总次数：9,894未命中次数：299未命中率：3.02%
替换次数：861

Cache大小均为8B，块大小均为64，文件选取047.tomcatv.din

1.相联度为1

Cache模拟结果：

总访问次数：1,000,000总未命中次数：18,393总未命中率：1.84%
指令访问：总次数：615,544未命中次数：2,411未命中率：0.39%
数据读：总次数：253,723未命中次数：4,543未命中率：1.79%
数据写：总次数：130,733未命中次数：11,439未命中率：8.75%
替换次数：18,265

2.相联度为2

Cache模拟结果：

总访问次数：1,000,000总未命中次数：16,550总未命中率：1.65%
指令访问：总次数：615,544未命中次数：1,322未命中率：0.21%
数据读：总次数：253,723未命中次数：2,943未命中率：1.16%
数据写：总次数：130,733未命中次数：12,285未命中率：9.40%
替换次数：16,422

3.相联度为4

Cache模拟结果：

总访问次数：1,000,000总未命中次数：10,332总未命中率：1.03%
指令访问：总次数：615,544未命中次数：130未命中率：0.02%
数据读：总次数：253,723未命中次数：687未命中率：0.27%
数据写：总次数：130,733未命中次数：9,515未命中率：7.28%
替换次数：10,204

4.相联度为8

Cache模拟结果：

总访问次数：1,000,000总未命中次数：9,708总未命中率：0.97%
指令访问：总次数：615,544未命中次数：286未命中率：0.05%
数据读：总次数：253,723未命中次数：614未命中率：0.24%
数据写：总次数：130,733未命中次数：8,808未命中率：6.74%
替换次数：9,580

相联度均为4，块大小均为64，文件选取022.li.din

1.Cache大小8B

Cache模拟结果：

总访问次数：1,000,003总未命中次数：11,591总未命中率：1.16%
指令访问：总次数：742,254未命中次数：5,389未命中率：0.73%
数据读：总次数：155,399未命中次数：4,007未命中率：2.58%
数据写：总次数：102,350未命中次数：2,195未命中率：2.14%
替换次数：11,463

2.Cache大小16B

Cache模拟结果：

总访问次数：1,000,003总未命中次数：4,581总未命中率：0.46%
指令访问：总次数：742,254未命中次数：1,410未命中率：0.19%
数据读：总次数：155,399未命中次数：1,928未命中率：1.24%
数据写：总次数：102,350未命中次数：1,243未命中率：1.21%
替换次数：4,325

3.Cache大小32B

Cache模拟结果：

总访问次数：1,000,003总未命中次数：2,249总未命中率：0.22%
指令访问：总次数：742,254未命中次数：582未命中率：0.08%
数据读：总次数：155,399未命中次数：731未命中率：0.47%
数据写：总次数：102,350未命中次数：936未命中率：0.91%
替换次数：1,737

4.Cache大小64B

Cache模拟结果：

总访问次数：1,000,003总未命中次数：1,555总未命中率：0.16%
指令访问：总次数：742,254未命中次数：461未命中率：0.06%
数据读：总次数：155,399未命中次数：264未命中率：0.17%
数据写：总次数：102,350未命中次数：830未命中率：0.81%
替换次数：598